

PCLMM Session 1

Foundations of DeFi and Automated Market Makers

Core Definitions

Decentralized Finance (DeFi) A financial system built on blockchain infrastructure that operates without central intermediaries like banks or brokerages.

Decentralized Exchange (DEX) A peer-to-peer marketplace where transactions occur directly between crypto traders through self-executing smart contracts.

Liquidity Pool A smart contract containing locked digital assets (the "inventory") supplied by users, which provides the necessary capital to facilitate trades on a DEX.

Market Makers Entities (traditionally institutions or high-frequency trading firms) that provide liquidity to a market by standing ready to buy and sell assets at all times, earning a spread or fee in return.

Automated Market Maker (AMM) An algorithmic protocol that replaces human market makers. It uses mathematical formulas embedded in smart contracts to automatically price assets based on supply and demand dynamics.

Uniswap V2 Model A standard AMM design that utilizes the constant product formula ($x \cdot y = k$). It forces deposited liquidity to be distributed uniformly across every possible price from zero to infinity.

V2 Capital Inefficiency The inherent flaw of the V2 model, particularly for stable or correlated assets. Because capital is spread infinitely thin, the vast majority (~99%) of a liquidity pool sits entirely idle at unrealistic price extremes, resulting in severe slippage for large trades.

The Need for V3 (Concentrated Liquidity) A next-generation AMM model that allows liquidity providers to "snipe" or concentrate their capital within specific, custom price bounds. This eliminates idle capital, providing up to 4000x capital leverage and drastically reducing slippage for traders.

Scenario A: Volatile Asset Swap (SOL/USDC)

1. Initial Pool State

The pool uses the constant product formula to maintain equilibrium:

$$k = x \cdot y$$

Where:

- $x = 100$ (SOL reserve)
- $y = 15000$ (USDC reserve)

$$k = 100 \times 15000 = 1500000$$

2. Swap Execution (Selling 10 SOL)

A user inputs a trade of $\Delta x = 10$ with a fee of **0.3%**. The effective input Δx_{eff} added to the pool's pricing curve is:

$$\Delta x_{eff} = 10 \cdot (1 - 0.003) = 9.97$$

The new pool reserves are calculated to maintain the invariant k :

$$\begin{aligned} x_{new} &= x + \Delta x_{eff} = 100 + 9.97 = 109.97 \\ y_{new} &= \frac{k}{x_{new}} = \frac{1500000}{109.97} \approx 13640.08 \end{aligned}$$

3. Output and Execution Price

The output Δy given to the user is the difference between the old and new USDC reserves:

$$\Delta y = y - y_{new} = 1500000 - 13640.08 = 1359.92$$

The effective execution price for the user is:

$$P_{exec} = \frac{\Delta y}{\Delta x} = \frac{1359.92}{10} = 135.99$$

Scenario B: The Stablecoin Whale Dump (USDC/USDT)

1. Initial Pool State

The stablecoin pool operates at a perfect 1:1 peg:

- $x = 1000000$ (USDC reserve)
- $y = 1000000$ (USDT reserve)

$$k = 1000000 \times 1000000 = 10^{12}$$

2. Swap Execution (Dumping 500,000 USDC)

A whale inputs $\Delta x = 500000$. Calculating the raw curve shift before applying the fee:

$$x_{new} = x + \Delta x = 1000000 + 500000 = 1500000$$
$$y_{new} = \frac{k}{x_{new}} = \frac{10^{12}}{1500000} \approx 666666.66$$

The raw output Δy_{raw} from the curve shift is:

$$\Delta y_{raw} = y - y_{new} = 1000000 - 666666.66 = 333333.33$$

Applying the **0.3%** fee to the final output:

$$\Delta y_{actual} = 333333.33 \cdot (1 - 0.003) \approx 332665$$

3. Slippage Calculation

Because the tokens should be pegged 1:1, the expected output was **500,000**. The slippage experienced by the whale is calculated as the ratio of actual output to expected output:

$$\text{Output Ratio} = \frac{332665}{500000} \approx 0.66533$$
$$\text{Price Slippage} = (1 - 0.66533) \cdot 100 \approx 33.5\%$$

Proving the "Idle Capital" Problem

A V2 liquidity pool holds 1,000,000 USDC and 1,000,000 USDT (stablecoins). Because they are pegged to the dollar, the price in reality only oscillates between \$0.99 and \$1.01.

The price in a V2 pool can be defined as $P = y/x$. Using the constant product formula, calculate the exact number of USDC tokens (x) remaining in the pool when the price hits the lower bound of \$0.99, and the upper bound of \$1.01.

Based on those bounds, roughly how many USDC tokens out of the original 1,000,000 are actually being used to facilitate trades within this realistic price range?

Substitute $y = P \cdot x$ into $x \cdot y = k$ to get $P \cdot x^2 = k$, which means $x = \sqrt{k/P}$. Here, $k = 1,000,000 \times 1,000,000 = 10^{12}$.

- **At P = 1.01:** $x = \sqrt{10^{12}/1.01} \approx 995,037$ USDC.
- **At P = 0.99:** $x = \sqrt{10^{12}/0.99} \approx 1,005,037$ USDC.

The x reserve only fluctuates between roughly 995,000 and 1,005,000. This mathematically proves that only about $\sim 5,000$ USDC (0.5% of the capital) is actively moving around the \$1 peg. The other 995,000 USDC is dead weight.

The Impermanent Loss Reality Check

You provide liquidity to an ETH/USDC V2 pool. You deposit 10 ETH and 20,000 USDC (Current ETH price is 2,000 USDC). The total value of your deposit is \$40,000.

A month later, the broader market price of ETH skyrockets to 4,000 USDC. Arbitrageurs trade with your pool until the internal pool price matches the 4,000 USDC market price.

Calculate the current total dollar value of your liquidity pool position. Then, calculate what your dollar value would have been if you had simply held your 10 ETH and 20,000 USDC in your wallet instead of providing liquidity. What is your net loss?

- **Initial State:** $x = 10$, $y = 20,000$. $k = 200,000$.
 - **New State (at $P = 4000$):** Using $x = \sqrt{k/P}$ and $y = \sqrt{k \cdot P}$:
$$x = \sqrt{200,000/4000} = \sqrt{50} \approx 7.071 \text{ ETH}$$
$$y = \sqrt{200,000 \cdot 4000} = \sqrt{800,000,000} \approx 28,284 \text{ USDC}$$
 - **Value in Pool:** $(7.071 \times 4000) + 28,284 = 28,284 + 28,284 = \$56,568$.
 - **Value if Held (HODL):** $(10 \times 4000) + 20,000 = \$60,000$.
 - **Impermanent Loss:** $60,000 - 56,568 = \$3,432$.
-

The V2 Sandbox Build

To truly understand these mechanics, you need to break them yourself. We highly encourage you to clone the Week 0 repository and experiment with the V2 MVP build. Try plugging in the scenarios from the questions above (the Whale Dump, the Stablecoin pool) and watch the constant product curve react in real-time.

GitHub Repository: <https://github.com/Singularity0/pclmm-v2-mvp>

How to run the simulator locally:

1. Clone the repository to your local machine.
2. In the root directory, start the Rust backend:

```
cargo run
```

3. Open a new terminal tab, navigate to the frontend folder, and start the React app:

```
cd frontend  
npm install  
npm run dev
```